48. Embeddings, Vector Search, Similarity

48. Embeddings, Vector Search, Similarity

Phase: Phase 7 — AI engineering in Go
Estimated time: 2 days
Slug: `48-embeddings-vectors`

Learning objectives

- Compute cosine similarity
- Normalize vectors
- Sketch top-K retrieval
- Understand ANN vs exact search

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Embeddings

Vectors representing text/images. Similarity ≈ semantic closeness.

```
func Cosine(a, b []float64) float64 {
    return Dot(a, b) / (Norm(a) * Norm(b))
}
```

Retrieval

Exact: scan all docs O(n·d). Approximate Nearest Neighbor (ANN) libraries trade recall for speed (HNSW, IVF).

Go's role

Go often owns the service: store vectors (pgvector/remote DB), retrieve top-K, pass chunks to the LLM.

Common mistakes (especially from Python instincts)

- Forgetting to normalize before dot-as-cosine
- Comparing vectors from different models
- n² algorithms on large corpora without ANN

Exercises

Work in `exercises/48-embeddings-vectors/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/48-embeddings-vectors/` until you have an honest failing attempt.

Self-check

1. Cosine vs dot product?
2. What is ANN?
3. Why model versioning matters?

Further reading

- https://en.wikipedia.org/wiki/Cosine_similarity